FINAL Report

Digital Theremin

a Full Implementation with Renessas MCU AIK-RA8D1

Team Lemon Tea

Pin-Jing, Li (111511015 *ouo.ee11@nycu.edu.tw)*

Ching-Kai, Huang (111511151 *kevin.ee11@nycu.edu.tw)*

Duan-Kai, Hu (112511015 *huduankai@gmail.com)*

January 5, 2026

Abstract

Theremin is an analog instrument that is notorious for it's expensive price and being hard to perform. Furthermore, the analog structure makes it harder to incoporate with the nowaday Digital Audio Workstations. To combat this, our group implemetned a digital version of this instrument, that is more affordable, easier to play, and compatible to digital audio workstations as a MIDI input device.

1 Introduction

I believe that our group is one of the only few groups giving a hoot about the assigned Renessas MCU as the main research focus. In this report, we will walk everyone through extensively about the details with the MCU, from hardware to firmware configurations and implementations. We hope that this documentation can also make it easier for not just the future students but also the future TAs when anyone is trying to perform any exploration with this module.

2 Circuit Deployment

Our implementation includes three main parts.

1 AIK-RA8D1 Development Board The core MCU of this experiment, responsible for control, signal acquisition, and data processing. Provides ADC pins (e.g., AN121, AN000, AN001) for analog signal sampling and transfers data to PC via USB CDC. Supports interfaces such as I^2C and UART for peripheral communication.

2 HC-SR04 Ultrasonic Module Transmitter (Tx): Triggered by a high-level signal (<10 μs), it emits 8 pulses at 40 kHz. Receiver (Rx): Captures the reflected signal, which is sampled by the MCU's ADC.

3 HP-122 Passive Buzzer Input a modulated PWM signal to control the frequency, and splitting voltage to control the volume.

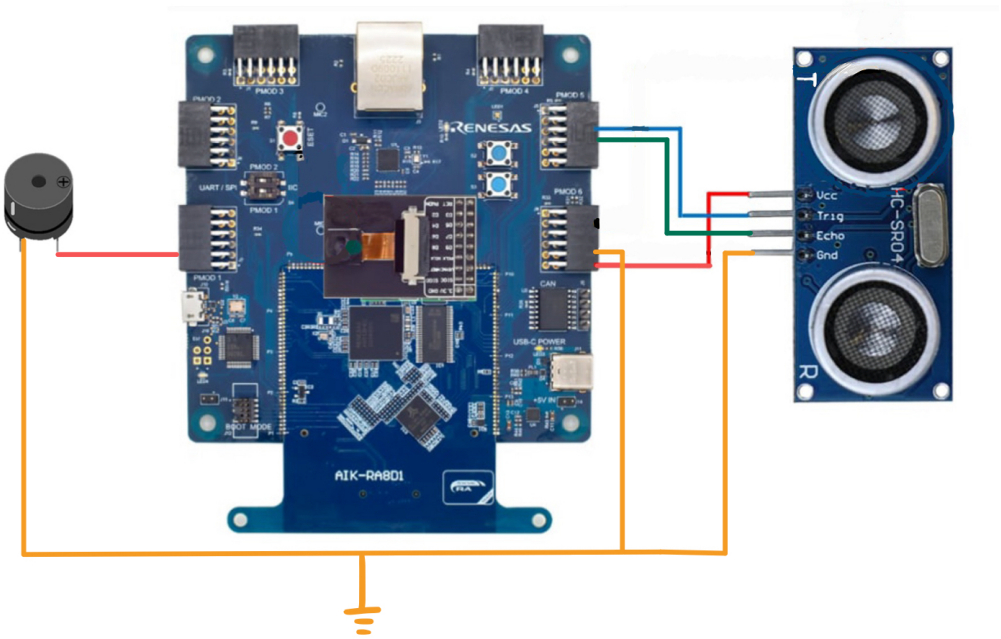
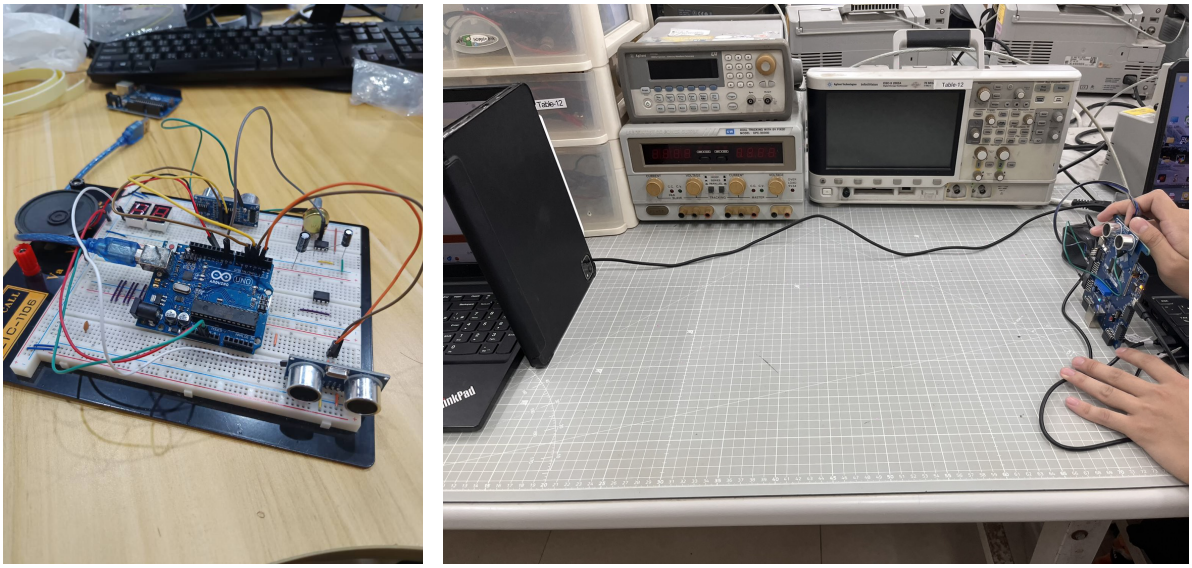


Figure 1: hardware configurations



(a) Circuit deployment on our beloved AIK-RA8D1.

(b) measurement experiments setups. We use our tablet as the reflection board, and the distance is referenced with the table pad.

Figure 2: hardware configurations

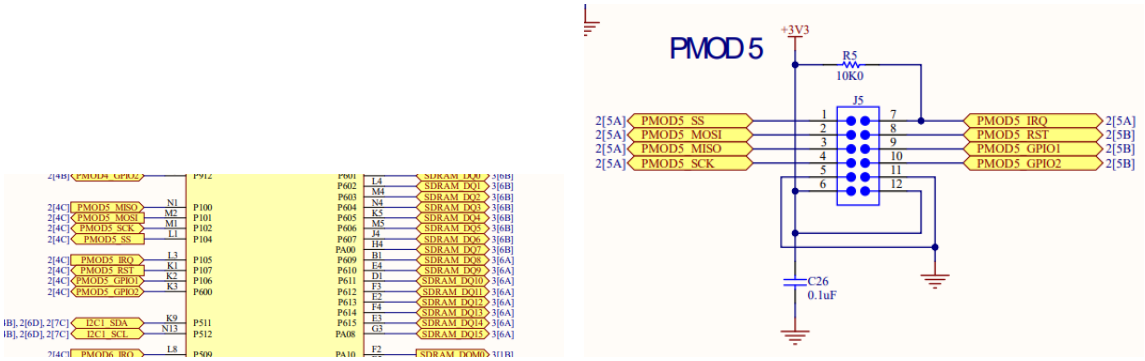
3 FPS configurations

To setup the MCU properly, we need to walk through a series of tracing. To start with, we need to first find the correct and compatible PIN that serves our desired purpose.

3.1 PIN Hunting Procedure

The following steps describe the systematic procedure used for PIN hunting:

1. In the user manual documentation, identify the PIN that supports the desired function we need (e.g. P102).
2. In the Daughterboard schematic, locate the corresponding symbolic name (e.g., PMOD5_SCK).
3. Use this symbolic name to search for the signal in the Motherboard schematic.
4. Check whether there are capacitors, resistors, or other components on the signal path that may affect signal integrity in the schematic.
5. Record the exact numbering on the board (e.g., J5-4).
6. Locate the correct physical pin on the actual hardware board.
7. Set up the hardware accordingly.



(a) Daughterboard Schematic, where we can find how the PIN correspond to the symbolic name in the Motherboard Schematic.

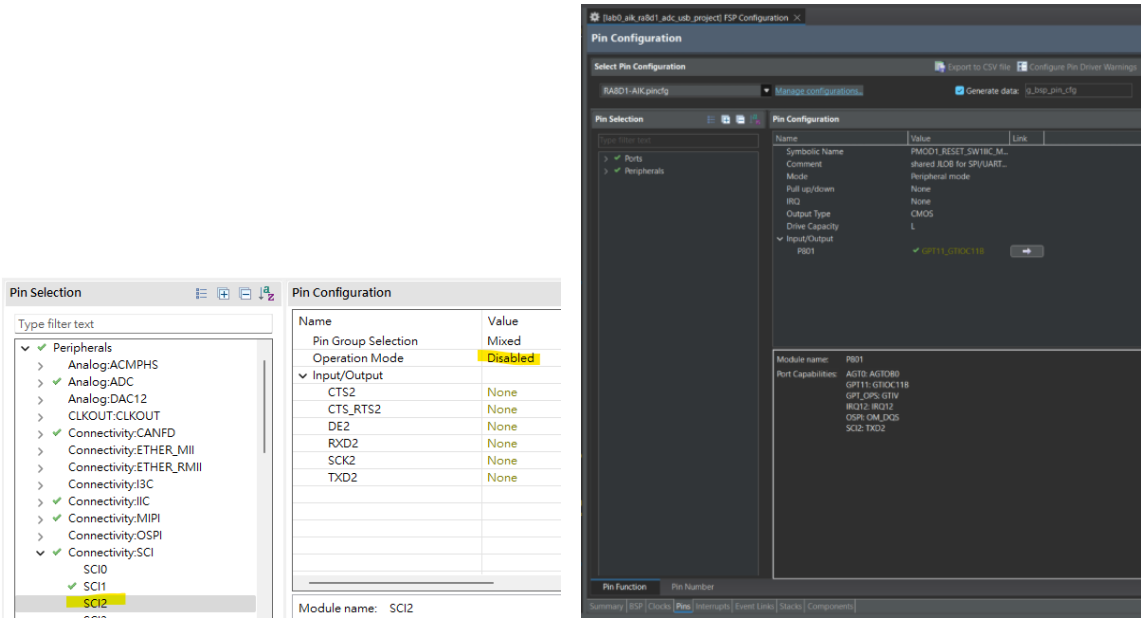
(b) Use the identified symbolic name to find the exact location on the board.

Figure 3: hardware configurations

3.2 FSP configuration

Since one single PIN usually support multiple functions, in the e^2 studio we need to set each of them to the actual function we expect them to work.

1. In the FSP Configuration tool, under the `Pins` tab, locate the desired PIN in the `Ports` list.
2. Check the capability of the selected port and configure it to the appropriate function.
3. If the PIN is already occupied, navigate to the `Peripherals` section and disconnect the original function.
4. Ensure that the original function is not required before reassignment.



(a) FSP Peripherals tab.

(b) FSP Pins tab.

Figure 4: FSP configurations

4 HC-SR04–Distance Measurement Module

4.1 Trigger Pin Configuration and Signal Specification

The trigger signal is generated by the MCU to provide a periodic TTL-level timing reference for external modules. This subsection describes the hardware configuration, timing source, and electrical and temporal specifications of the trigger pin.

4.1.1 Pin Assignment

The trigger output is assigned to pin `P100`. This pin is configured as a GPIO output under the Renesas Flexible Software Package (FSP) pin configuration interface.

- **Pin**: `P100`
- **Direction**: Output
- **Output type**: CMOS
- **Logic level**: TTL-compatible

The output polarity is configured as *initial low*, and the pin is driven directly by the timer output logic without additional hardware gating.

4.1.2 Timer Source: Asynchronous General Timer (AGT)

The trigger signal is generated using the AGT, a low-power timer peripheral designed to operate independently of the high-speed system clock. It is suitable for periodic signal generation with minimal power consumption.

- **Timer module**: AGT

- **Channel**: 0
- **Counter width**: 16-bit
- **Maximum count value**: $2^{16} = 65536$

4.1.3 Clock Source and Timing Resolution

The AGT is driven by a low-power clock source. This clock determines the fundamental timing resolution of the trigger signal.

- **Clock type**: Low-power clock
- **Effective clock frequency**: Set to 27 kHz in FSP

All timing parameters (pulse width and period) are derived from this clock frequency.

4.1.4 Trigger Signal Timing Specification

The trigger pin outputs a periodic TTL pulse with the following characteristics:

- **Pulse width**: 10 μs
- **Repetition rate**: approximately 26.67 kHz

The pulse width is defined by the AGT compare match value, while the period is defined by the timer overflow value.

A timing diagram of the trigger signal is illustrated conceptually as:

`HIGH (10 µs)` $\rightarrow$ `LOW` $\rightarrow$ repeat

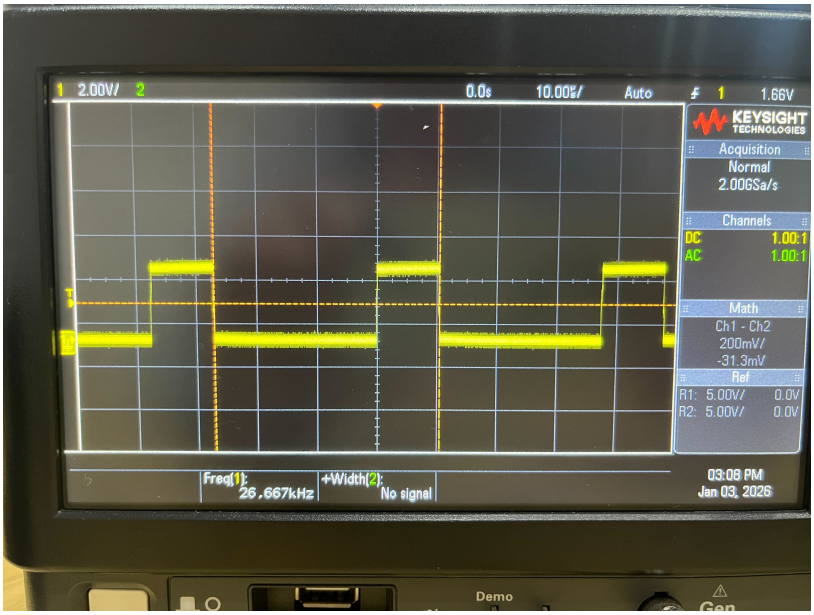


Figure 5: The signal Output by P100. This is served as the input signal to the Trigger pin. We can see that

4.2 Echo Pin Configuration and Measurement Principle

The echo pin is used to receive a TTL-level pulse whose duration encodes the round-trip propagation time of the transmitted signal. This subsection describes the timer configuration, edge detection strategy, and distance computation method.

4.2.1 Pin and Timer Assignment

The echo signal is connected to pin `P102`, which is configured as a timer input channel.

- **Pin**: `P102`
- **Timer module**: General PWM Timer (GPT)
- **Channel**: GPT2, GTIO Channel 2B
- **Mode**: Periodic
- **Counter width**: 16-bit
- **Period**: `0x10000` (raw counts)

The GPT counter runs continuously and provides a free-running time base for pulse width measurement.

4.2.2 Edge Detection and Time Measurement

To measure the pulse width of the echo signal, both edges of the input waveform are detected:

- **Rising edge**: marks the start of the echo pulse
- **Falling edge**: marks the end of the echo pulse

The timer count values are captured at each edge. The difference between the captured counts corresponds to the pulse duration in timer ticks.

4.2.3 Echo Pulse Interpretation

The echo pin receives a TTL-level pulse whose high-level duration is proportional to the signal's round-trip time of flight. Conceptually,

$$\texttt{Pulse Width} \propto \texttt{Propagation Time}$$

This pulse is generated by the external module and fed directly into the GPT input capture logic.

4.2.4 Distance Conversion

Let x denote the measured pulse width in microseconds. The corresponding distance is computed as

$$\text{Distance (cm)} = \frac{x}{58}$$

This conversion factor accounts for the round-trip travel time of the signal and the propagation speed in air.

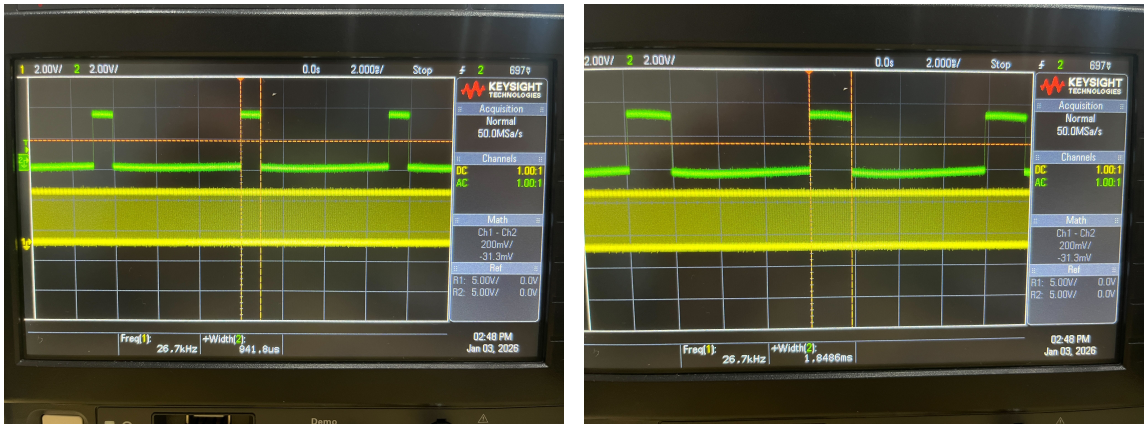
4.2.5 Functional Role of the Echo Pin

The echo pin serves as the timing measurement input that:

- captures the duration of the returned signal,
- enables distance estimation based on time-of-flight,
- operates independently of the trigger generation logic.

Using GPT input capture ensures accurate edge-aligned timing measurement with minimal software overhead.

Figure 6: The signal Output by echo pin. The green curve is the PWM signal carrying the ToF with distance $d = 5cm$, and the yellow curve is the trigger pin signal. This in



(a) 15cm. ToF = 941.8μs (b) 30cm. ToF = 1848.6μs

Figure 7: Echo Pin output signal with different distance. The green wave is the square pulse sent by echo pin, and the yellow wave is the trigger pin signal. This indicates that the Module is functioning as expected, all the configuration has been properly set.

4.2.6 Callback Function

We are now able to finally implement how we want our board to react once they've detected the expected rising/falling signal from the echo pin, and convert it back to distance.

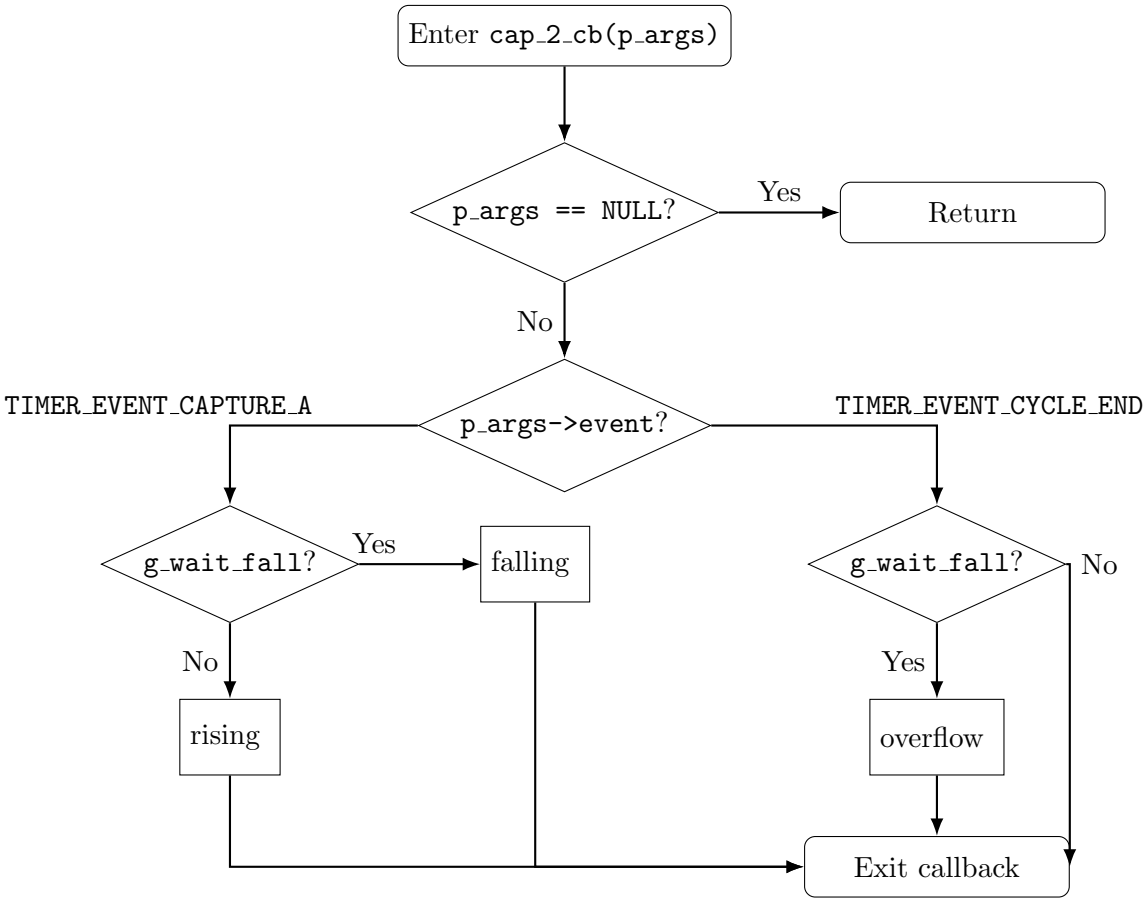


Figure 8: Control flow of the GPT2 input-capture callback for echo pulse measurement.

Listing 1: GPT2 Input Capture Callback for Echo Pulse Measurement

```
void cap_2_cb(timer_callback_args_t *p_args)
{
    if (NULL == p_args)
    {
        return;
    }

    switch (p_args->event)
    {
        case TIMER_EVENT_CAPTURE_A:
        {
            if (!g_wait_fall)
            {
                /* First edge: treat as rising */
                g_echo_rise = (uint32_t) p_args->capture;
                g_capture_overflow = 0;
                g_wait_fall = true;
            }
            else
            {
                /* Second edge: treat as falling */
                g_echo_fall = (uint32_t) p_args->capture;
                g_wait_fall = false;
```

```
                b_start_measurement = true;
            }
            break;
        }

        case TIMER_EVENT_CYCLE_END:
        {
            /* Timer wrapped between rise and fall */
            if (g_wait_fall)
            {
                g_capture_overflow++;
            }
            break;
        }

        default:
            break;
    }
}
```

5 HPE-122 Passive Buzzer

We choose a cheap passive buzzer as our audio output module since we are familiar with it.

5.1 FPS configuration

The buzzer is driven by a hardware PWM signal generated using the General PWM Timer (GPT). This subsection describes the pin assignment, timer configuration, and achievable output frequency range.

5.1.1 Pin and Timer Assignment

The buzzer output is mapped to pin `P801`, which is configured as a GPT output channel.

- **Pin**: `P801`
- **Timer module**: GPT
- **Channel**: GPT11
- **Output channel**: GTIO Channel 11B

5.1.2 PWM Configuration

The GPT is configured in saw-wave PWM mode to generate a square-wave signal suitable for driving a buzzer.

- **PWM mode**: Saw-wave PWM
- **Output channel enabled**: Channel B
- **Duty cycle**: 50%

- **Initial output level**: Low

Channel A output is disabled, and only Channel B is used to simplify signal routing.

5.1.3 Operating Frequency

The PWM output frequency is set to approximately 1 kHz, which lies within the audible range and is appropriate for buzzer operation.

5.1.4 Allowed Output Frequency Range

The GPT operates with the following constraints:

- **Clock frequency**: 60 MHz
- **Counter width**: 16-bit

The minimum achievable output frequency is therefore limited by the maximum counter period:

$$f_{\text{min}} = \frac{60\ \text{MHz}}{2^{16}} \approx 915\ \text{Hz}$$

The actual timer parameters and clock frequency are verified at runtime using `R_GPT_InfoGet`.

5.1.5 Functional Role

The buzzer signal provides audible feedback to the user. Its frequency and duty cycle are controlled entirely in hardware, ensuring stable operation with minimal CPU involvement.

(a) 10cm, 1.2598kHz (b) 15cm, 1.3349kHz

Figure 9: Frequency change of the PWM signal (yellow wave). We can observe that different distance calculated with the echo pin signal (green wave) is inducing different frequency accordingly.

Listing 2: PWM Configuration Function for Buzzer Output

```
static fsp_err_t buzzer_pwm_set(float freq_hz, float duty_percent)
{
    /* Duty-cycle clamping */
    if (freq_hz < 1.0f) { freq_hz = 1.0f; }
```

```
    if (duty_percent < 0.0f) { duty_percent = 0.0f; }
    if (duty_percent > 100.0f) { duty_percent = 100.0f; }

    timer_info_t info = {0};
    fsp_err_t err = R_GPT_InfoGet(&g_timer11_ctrl, &info);
    if (FSP_SUCCESS != err) { return err; }

    uint32_t clk = info.clock_frequency;
    if (clk == 0U) { return FSP_ERR_INVALID_ARGUMENT; }

    /* Minimum frequency clamp due to 16-bit period limit */
    float f_min_hw = (float) clk / (float) (GPT11_MAX_PERIOD_COUNTS + 1U);
    if (freq_hz < f_min_hw)
    {
        freq_hz = f_min_hw;
    }

    /* Compute timer period from desired frequency */
    float ticks_per_cycle_f = (float) clk / freq_hz;
    if (ticks_per_cycle_f < 2.0f)
    {
        ticks_per_cycle_f = 2.0f;
    }

    uint32_t ticks_per_cycle = (uint32_t) (ticks_per_cycle_f + 0.5f);
    if (ticks_per_cycle < 2U) { ticks_per_cycle = 2U; }

    uint32_t period_counts = ticks_per_cycle - 1U;
    if (period_counts > GPT11_MAX_PERIOD_COUNTS)
    {
        period_counts = GPT11_MAX_PERIOD_COUNTS;
    }

    /* Duty-cycle conversion */
    uint32_t duty_counts =
        (uint32_t) ((float) (period_counts + 1U) * (duty_percent / 100.0f));

    if (duty_counts == 0U)
    {
        duty_counts = 1U;
    }
    if (duty_counts >= (period_counts + 1U))
    {
        duty_counts = period_counts;
    }

    /* Apply configuration */
    (void) R_GPT_Stop(&g_timer11_ctrl);

    err = R_GPT_PeriodSet(&g_timer11_ctrl, period_counts);
    if (FSP_SUCCESS != err) { return err; }

```

```
    err = R_GPT_DutyCycleSet(&g_timer11_ctrl,
                             duty_counts,
                             GPT_IO_PIN_GTIOCB);
    if (FSP_SUCCESS != err) { return err; }

    return R_GPT_Start(&g_timer11_ctrl);
}
```

6 Main Control Flow Implementation

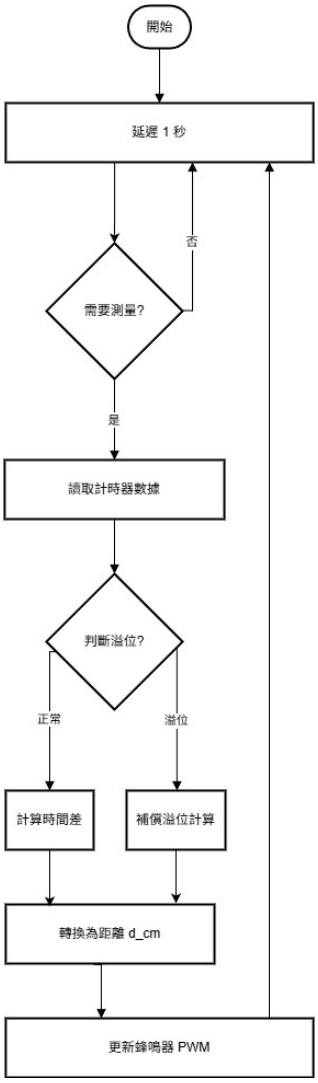


Figure 10: Control flow of the main function

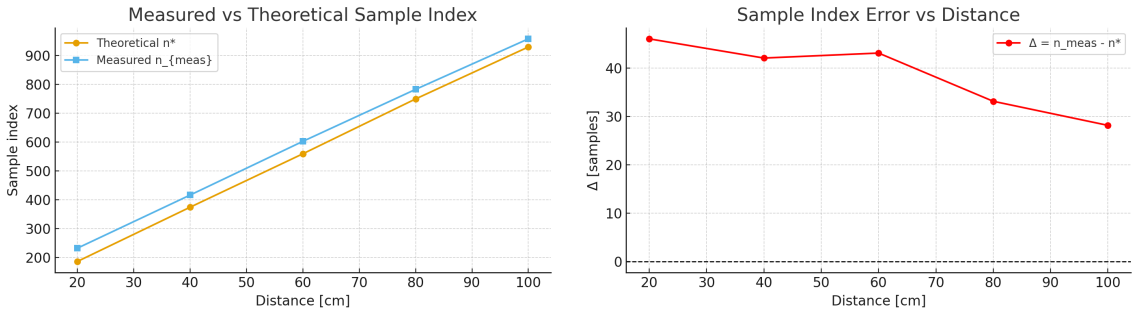
7 Distance Error Source Analysis

We perform a similar error calibration as in the previous experiment. Firstly, since we're using the same module, we reconducted the distance measurement comparing to the theoretical transformation equations.

Pair (cm)	n^*_{theory}	n_{meas}	$\Delta n^*_{\text{theory}}$	Δn_{meas}	$\frac{\Delta n_{\text{meas}}}{\Delta n^*_{\text{theory}}}$	Relative Error (%)
20→40	184.97→369.94	231→412	184.97	181	0.979	−2.1
40→60	369.94→554.91	412→598	184.97	186	1.006	+0.6
60→80	554.91→739.88	598→773	184.97	175	0.946	−5.4
80→100	739.88→924.86	773→953	184.97	180	0.973	−2.7

Table 1: Relative Distance Comparison (Measured vs. Theoretical)

The measured peak indices n_{meas} were aligned relative to the start of transmission, determined by the maximum slope of the Tx waveform. We then plotted n_{meas} against n^*, as well as the corresponding sample index error $\Delta n = n_{\text{meas}} - n^*$, as shown below.



(a) Measured peak index and theoretical n^* (b) Sample index error vs. distance

Figure 11: Comparison between measured peak indices and theoretical sample indices.

From Fig. 11b, we observe that the error Δn decreases approximately linearly as distance increases, consistent with a fixed system delay plus a small distance-dependent effect. Instead of the usual choice of a constant model, we found out that the error is related to the distance. We therefore model the error as

$$\Delta n = a + b\, n_{\text{meas}},$$

and use linear regression to estimate $a \simeq 53.14$ and $b \simeq -0.02469$. Applying this model, we compute corrected sample indices $\hat{n} = n_{\text{meas}} - \Delta n$, yielding the results summarized below.

True Distance (cm)	n_{meas}	$\hat{n}$ (Corrected)	n^* (Theory)	Error After Correction (cm)
20	231	183.56	184.97	−0.15
40	412	369.04	369.94	−0.10
60	598	559.39	554.91	+0.48
80	773	738.65	739.88	−0.13
100	953	922.23	924.86	−0.28

Table 2: Corrected Sample Index and Residual Distance Error (Linear Model)

The corrected indices significantly reduce the bias, with all residual distance errors falling within ±0.5 cm, demonstrating that the linear error model is effective.

To translate the measured sample index back into a physical distance, we use the relation between the sample index n, the sampling frequency F_s, and the speed of sound v. Each sample corresponds to a time increment of

$$T_s = \frac{1}{F_s},$$

so the round-trip time corresponding to n samples is

$$t = n\,T_s = \frac{n}{F_s}.$$

Assuming a two-way (Tx–Rx) propagation path, the one-way distance is then given by

$$\hat{d} = \frac{v\,t}{2} = \frac{v}{2F_s}\,n.$$

For our experiment at $T = 25°\text{C}$, we use $v = 331 + 0.6T \approx 346$ m/s $= 34{,}600$ cm/s and $F_s = 160$ kHz, which yields the conversion factor

$$k = \frac{v}{2F_s} = \frac{34{,}600}{2 \times 160{,}000} \simeq 0.1081\ \frac{\text{cm}}{\text{sample}}.$$

Thus, the estimated distance from a measured sample index n_{meas} (or its corrected version $\hat{n}$) is

$$\hat{d} = k\,n.$$

This linear relation allows us to directly convert the corrected sample indices from Table 2 into absolute distance estimates and to compute residual errors in centimeters for performance evaluation.

To fully exploit our calibration, we convert our measured width to corresponding number of samples using $F_s = 160kHz$

distance(cm)	width(μs)	sample(n)	$\hat{d}$(cm)
5	327.5	52	5.66
10	620.8	99	10.74
15	941.4	151	16.28
20	1205.5	193	20.85
23	1240.2	198	21.45
30	1848.9	296	31.97

Table 3: Relative Distance Comparison (Measured vs. Theoretical)

8 Music Scale Discretization

We initially try to make the scale flexible as the true Theremin, but we found out that it will make it so hard to play and we really don't have that much of time to practice playing such a naive instrument. Therefore, we decided to make the scale discrete to give us some flexibility.

Listing 3: Distance-to-Frequency Mapping Quantized to C Major Scale

```
static float distance_to_freq_hz(float d_cm)
{
    /* Continuous inverse mapping */
    d_cm = clampf(d_cm, D_MIN_CM, D_MAX_CM);
    float inv = 1.0f / d_cm;
    float inv_min = 1.0f / D_MAX_CM;
    float inv_max = 1.0f / D_MIN_CM;

    float t = (inv - inv_max) / (inv_min - inv_max);
    t = clampf(t, 0.0f, 1.0f);

    float f_cont = F_MIN_HZ + (F_MAX_HZ - F_MIN_HZ) * t;

    /* Quantize to C major scale between F_MIN_HZ and F_MAX_HZ */
    const float f_base = F_MIN_HZ;
    float best_f = F_MIN_HZ;
    float best_err = 1e30f;

    for (int octave = 0; octave < 8; ++octave)
    {
        int octave_offset = 12 * octave;

        for (int i = 0; i < 7; ++i)
        {
            int semitone = octave_offset + k_major_steps[i];
            float f = f_base * powf(2.0f, (float) semitone / 12.0f);

            if (f < F_MIN_HZ) { continue; }
            if (f > F_MAX_HZ) { break; }

            float err = fabsf(f - f_cont);
            if (err < best_err)
            {
                best_err = err;
                best_f = f;
            }
        }
    }

    return best_f;
}
```

9 Digital Theremin

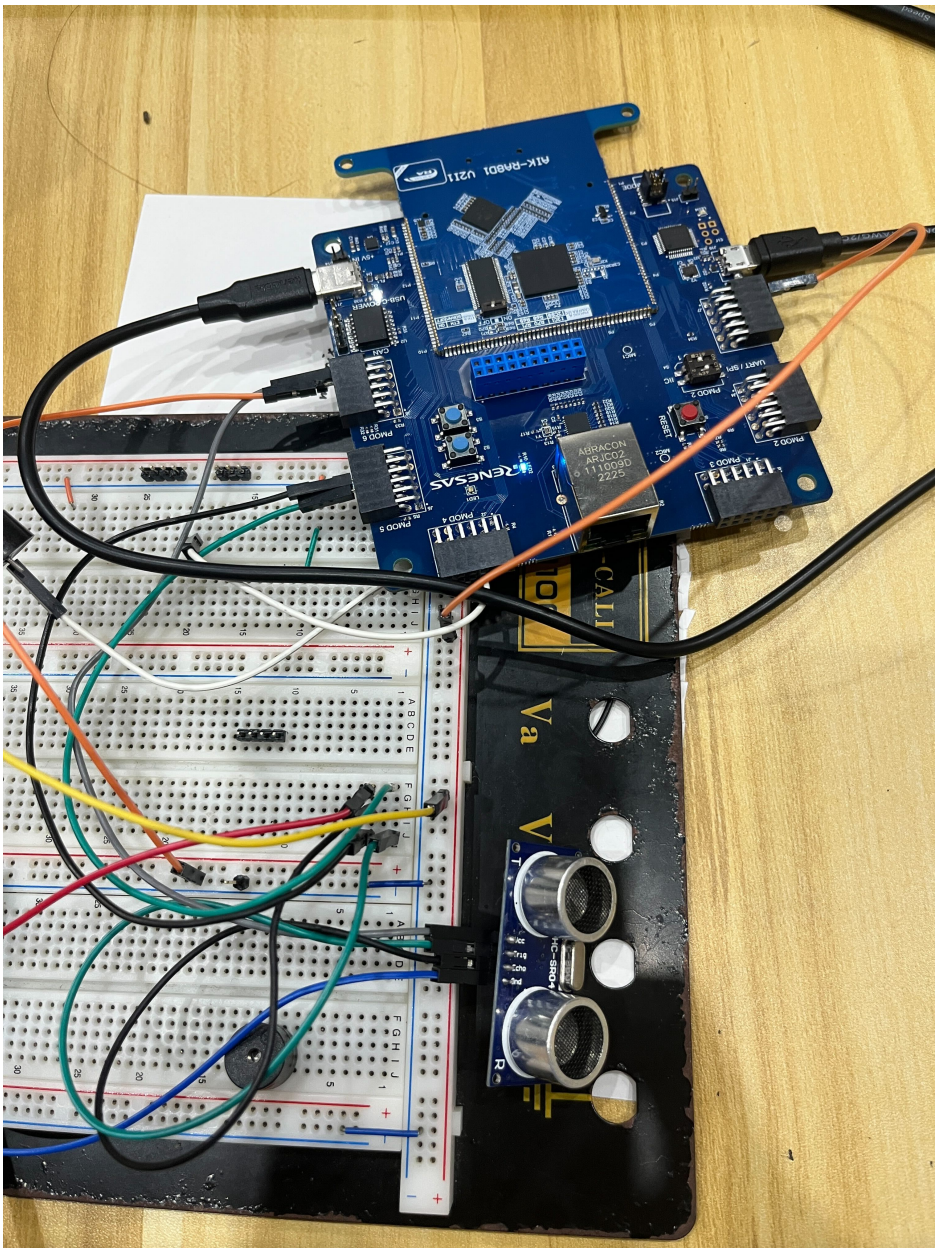


Figure 12: Final Implementation. We are really proud of actually making it work.

10 Quantized Scale to MIDI and PC Logging

This section describes how the quantized pitch (C major scale) can be converted into MIDI note messages and saved on a host computer as a standard MIDI file (`.mid`). The implementation is split into two parts: (i) generating MIDI events on the MCU and streaming them to the host, and (ii) recording the stream and writing a MIDI file on the PC.

10.1 MCU-Side: Frequency-to-MIDI and Serial Streaming

10.1.1 Mapping Quantized Frequency to a MIDI Note Number

After `distance_to_freq_hz()` returns a quantized frequency f (in Hz), the corresponding MIDI note number is computed using the equal-tempered formula

$$n = \text{round}\left(69 + 12\log_2\left(\frac{f}{440}\right)\right),$$

where MIDI note 69 corresponds to A4 = 440 Hz.

10.1.2 MIDI Message Format

A MIDI Note On event is a 3-byte message:

`0x90`, `note`, `velocity`

and Note Off can be sent as either `0x80` or `0x90` with velocity 0:

`0x90`, `note`, `0x00`.

In this project, we transmit raw MIDI bytes over a serial link (USB CDC or UART). The host PC interprets the incoming bytes and logs them into a MIDI file.

10.1.3 Reference Implementation (MCU)

Listing 4: MCU-side: Convert Frequency to MIDI Note and Stream Note Events

```c
#include <math.h>
#include <stdint.h>
#include <stdbool.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

static inline float log2f_compat(float x)
{
    return logf(x) / logf(2.0f);
}

static inline uint8_t clamp_u8(int v)
{
    if (v < 0) return 0;
    if (v > 127) return 127;
    return (uint8_t) v;
}

static uint8_t freq_to_midi_note(float f_hz)
{
    float n = 69.0f + 12.0f * log2f_compat(f_hz / 440.0f);
    int note = (int) lroundf(n);
    return clamp_u8(note);
```

```c
}

static void midi_stream_write_bytes(const uint8_t *buf, uint32_t len);

static void midi_note_on(uint8_t note, uint8_t velocity)
{
    uint8_t msg[3] = { 0x90, note, velocity };
    midi_stream_write_bytes(msg, 3);
}

static void midi_note_off(uint8_t note)
{
    uint8_t msg[3] = { 0x90, note, 0x00 };
    midi_stream_write_bytes(msg, 3);
}

static uint8_t g_last_note = 0xFF;

void buzzer_note_update_from_distance(float d_cm)
{
    float f_hz = distance_to_freq_hz(d_cm); /* quantized to C major */
    uint8_t note = freq_to_midi_note(f_hz);

    if (note != g_last_note)
    {
        if (g_last_note != 0xFF)
        {
            midi_note_off(g_last_note);
        }

        midi_note_on(note, 90);

        g_last_note = note;
    }
}
```

The function `buzzer_note_update_from_distance()` can be called at the same point where PWM frequency is updated. Instead of generating sound purely via PWM, the MCU emits MIDI note events that can be synthesized or recorded on the host.

10.2 PC-Side: Capture Stream and Save to a MIDI File

10.2.1 Design

The host PC reads the raw MIDI bytes coming from the serial port and reconstructs note events. These events are timestamped and stored into a standard MIDI track, then written to `output.mid`. This allows later playback in any DAW or MIDI player.

10.2.2 Reference Implementation (Python)

The following script uses:

- `pyserial` for reading from a serial device, and

- `mido` for writing a MIDI file.

Listing 5: PC-side: Read MIDI Bytes from Serial and Save as `.mid`

```
import time
import serial
import mido

PORT = "COM5"
OUT_MID = "output.mid"

# Simple 3-byte MIDI parser for Note On/Off on channel 1
def read_midi_messages(ser):
    buf = bytearray()
    while True:
        data = ser.read(ser.in_waiting or 1)
        if data:
            buf.extend(data)

        # Parse 3-byte messages: status, note, vel
        while len(buf) >= 3:
            status, note, vel = buf[0], buf[1], buf[2]
            if status in (0x90, 0x80):
                yield (status, note, vel)
                del buf[:3]
            else:
                # Resync: drop one byte
                del buf[0]

def main():
    ser = serial.Serial(PORT, BAUD, timeout=0.01)

    mid = mido.MidiFile(ticks_per_beat=480)
    track = mido.MidiTrack()
    mid.tracks.append(track)

    tempo = mido.bpm2tempo(120) # 120 BPM
    track.append(mido.MetaMessage("set_tempo", tempo=tempo, time=0))

    last_t = time.time()

    print("Recording... Press Ctrl+C to stop.")
    try:
        for status, note, vel in read_midi_messages(ser):
            now = time.time()
            dt_sec = now - last_t
            last_t = now

            # ticks = dt_sec * (ticks_per_beat * beats_per_second)
            bps = 120 / 60.0
            ticks = int(dt_sec * mid.ticks_per_beat * bps)

            if status == 0x90 and vel > 0:
```

```
                msg = mido.Message("note_on", note=int(note), velocity=int(vel),
                    time=ticks)
            else:
                msg = mido.Message("note_off", note=int(note), velocity=0, time=
                    ticks)

            track.append(msg)

    except KeyboardInterrupt:
        pass
    finally:
        ser.close()

    mid.save(OUT_MID)
    print(f"Saved: {OUT_MID}")

if __name__ == "__main__":
    main()
```

This logger produces a valid MIDI file that preserves the temporal structure of the note events emitted by the MCU. The resulting `output.mid` can be imported into any MIDI-capable audio environment for playback or further analysis.

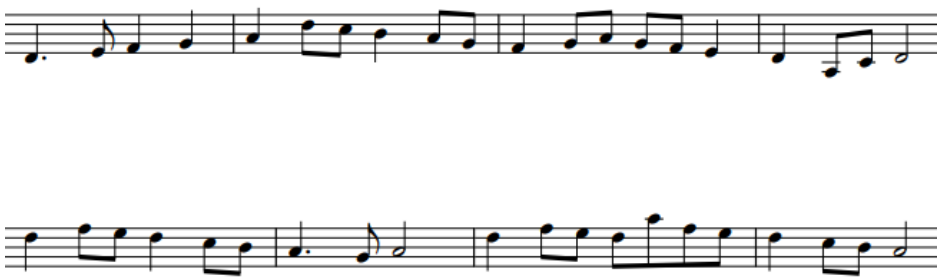


Figure 13: The Exported MIDI file opened in Musescore4

11 Acknowlegdement

We appreciate all the support from the Renessas Team, espectially the engineers Steven and Henry, and thanks to Hedy for the effective and responsive communication and schedule coordination. Also, thanks for all the TAs for your kind support. Through all the rough time trying to make this project work, we are always grateful for Prof. Jeng always being presence and giving us emotional support.